

Heaven's Light is Our Guide



Rajshahi University of Engineering & Technology (RUET)

Department of Electrical and Computer Engineering

Lab report-01

Course title: Digital Techniques Sessional

Course Code: ECE 2112

Submission Date: 2 August, 2026

<u>Submitted By</u> MD. MUSFIK AL-RAHIN ROKTIM Roll:2410058	<u>Submitted To</u> MST. Mazeda Noor Tasnim Lecturer Dept. of ECE, RUET
---	---

Topic Name: Introduction to Digital Logics

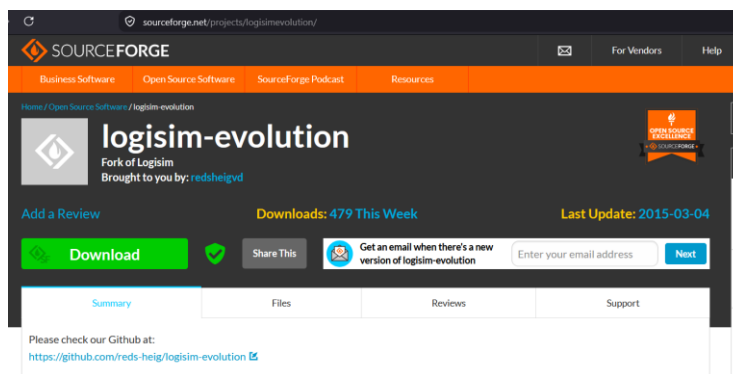
Logisim:

Logisim-evolution is a free, open-source educational software used for designing, simulating, and analyzing digital logic circuits. As the modern, actively maintained successor to the original Logisim, it provides an intuitive drag-and-drop workspace for building everything from simple gate schematics to complex CPUs, memory units, and 7400-series TTL components. In addition to real-time signal propagation and timing-waveforms (chronograms), it bridges theoretical logic design with real-world computer architecture by offering VHDL/Verilog code generation and direct board synthesis for continuous deployment onto FPGAs.

How to Download:

1. Visit SourceForge Page:

Go to the official **Logisim-evolution SourceForge page** at sourceforge.net/projects/logisim-evolution/.



2. Download the File:

Click the green **Download** button to automatically grab the latest installer for your operating system, or click the **Files** tab to select a specific version (e.g., .msi for Windows, .dmg for macOS, .deb for Linux).

3. Run the Installer:

Open the downloaded file and follow the standard on-screen installation prompts to launch Logisim-evolution.

Uses Of Logisim:

After the finish of downloading it will appear in the desktop with below icon.



To use it :

Place Components: Drag any Gate onto the canvas, place 2 Input Pins to its left, and 1 Output Pin to its right.

Connect Wires: Click and drag wires to connect the input pins to the gate inputs, and the gate output to the output pin.

Simulate: Select the Poke Tool, click the inputs to toggle 0 and 1, and watch the output update in real time.

Benefits of Logisim:

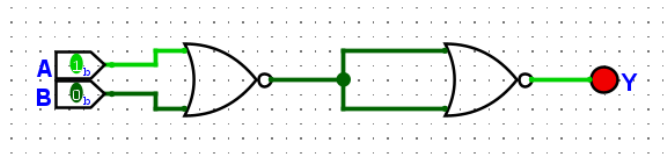
- **What it is:** A free, open-source GUI tool for designing, testing, and simulating digital logic circuits.
- **Core Advantage:** Simple drag-and-drop circuit building with real-time signal propagation and live interactive testing.
- **Standout Feature:** Direct export to VHDL/Verilog code and built-in synthesis support to run designs on real FPGA boards.
- **Key Tools:** Features an integrated chronogram (timing waveform analyzer), 7400-series TTL chips, and modular sub-circuit design.
- **Main Purpose:** Bridging classroom logic theory with practical computer architecture and processor design.

Experiment No 1: Introduction of Universal Logic Gates

NOR Gate:

NOR gate (NOT-OR) is a universal digital logic gate that produces a High output (1) only when all of its inputs are Low (0). If any input is High (1), the output goes Low (0). It is essentially an OR gate followed by an inverter (NOT gate).

Creating OR Gate using NOR Gate:



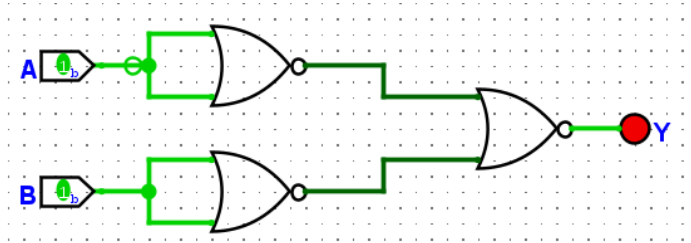
Truth Table:

Input A	Input B	Output
0	0	OFF
0	1	ON
1	0	ON
1	1	ON

Conclusion:

Building an OR gate using only NOR gates proves that the NOR gate is universal, meaning it can recreate any basic logic function on its own. By connecting a second NOR gate as an inverter, double negation cancels out the initial inversion ($\overline{\overline{A + B}} = A + B$), yielding standard OR logic while keeping chip manufacturing simple and cheap.

Creating AND Gate using NOR Gate:



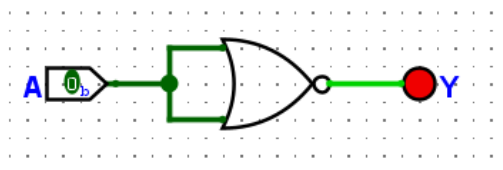
Truth Table:

Input A	Input B	Output
0	0	OFF
0	1	OFF
1	0	OFF
1	1	ON

Conclusion:

Building an AND gate using NOR gates proves the NOR gate's universality via De Morgan's Law ($\overline{\overline{A} + \overline{B}} = A \cdot B$). Inverting inputs A and B before passing them to a third NOR gate generates standard AND logic, allowing entire chips to be built using a single, cost-effective gate type.

Creating NOT Gate using NOR Gate:



Truth Table:

Input A	Output
0	ON
1	OFF

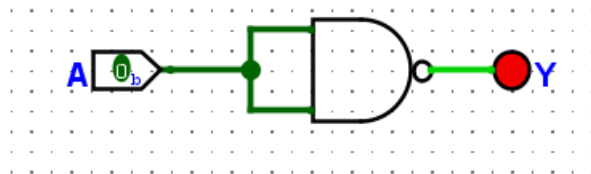
Conclusion:

Implementing a NOT gate using a NOR gate is the simplest proof of NOR gate universality. By tying both inputs together, the NOR gate's internal OR operation vanishes ($\overline{A + A} = \bar{A}$), directly producing the inverted output and making it the foundation for building all other logic gates.

NAND Gate:

A NAND gate (NOT-AND) is a universal digital logic gate that produces a Low output (0) only when all of its inputs are High (1), and a High output (1) in all other cases. Represented by the Boolean equation $Y = \overline{A \cdot B}$, it effectively combines an AND gate followed by an inverter. Because it is a universal gate—meaning any digital logic system, processor, or memory chip can be constructed using only NAND gates—it serves as the foundational building block of modern semiconductor and microchip architecture.

Creating NOT Gate using NAND Gate:



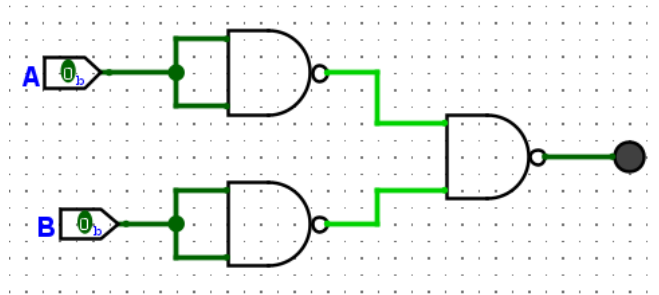
Truth Table:

Input A	Output
0	ON
1	OFF

Conclusion:

Creating a NOT gate using a NAND gate serves as the primary proof of NAND gate universality. By tying both inputs together, the internal AND function simplifies ($\overline{A \cdot A} = \bar{A}$), allowing a single NAND gate to directly invert a signal and act as the core building block for all other digital logic circuits.

Creating OR Gate using NAND Gate:



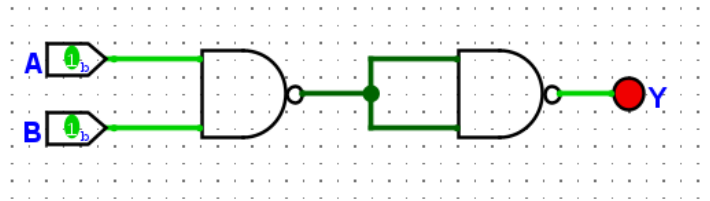
Truth Table:

Input A	Input B	Output
0	0	OFF
0	1	ON
1	0	ON
1	1	ON

Conclusion:

Constructing an OR gate using NAND gates validates De Morgan's Law ($\overline{\bar{A} \cdot \bar{B}} = A + B$) and confirms the NAND gate's universality. Inverting inputs A and B individually before feeding them into a third NAND gate yields standard OR logic, allowing engineers to design complete logic networks using a single, cost-effective gate type.

Creating AND Gate using NAND Gate:



Truth Table:

Input A	Input B	Output
0	0	OFF
0	1	OFF
1	0	OFF
1	1	ON

Conclusion:

Building an AND gate using NAND gates demonstrates the double negation property ($\overline{\overline{A \cdot B}} = A \cdot B$) and reconfirms NAND universality. By passing inputs through an initial NAND gate and inverting the result with a second NAND gate, standard AND logic is restored, simplifying integrated circuit design with uniform gate components.

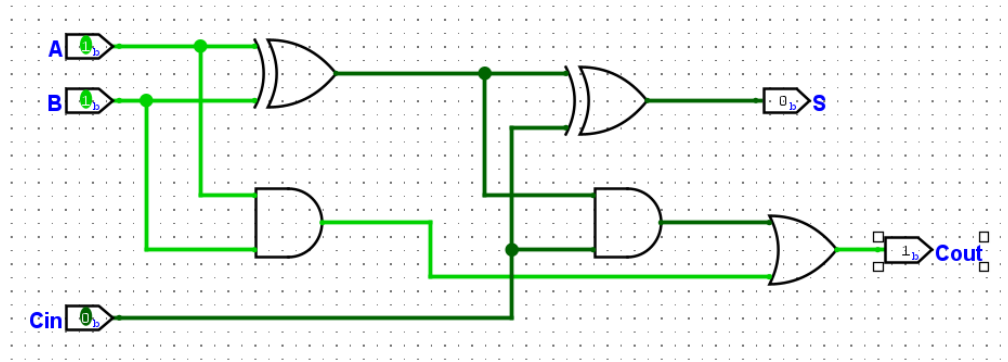
Adder:

An adder is a fundamental digital logic circuit designed to perform addition on binary numbers. It forms the core component of the Arithmetic Logic Unit (ALU) inside microprocessors, CPUs, and digital signal processors (DSPs), where it performs basic arithmetic operations as well as memory address calculation and table indexing.

Full-Adder:

Adds three single-bit inputs (A , B , and an incoming carry C_{in}) to produce Sum (S) and Carry (C_{out}). This is the building block for multi-bit adders.

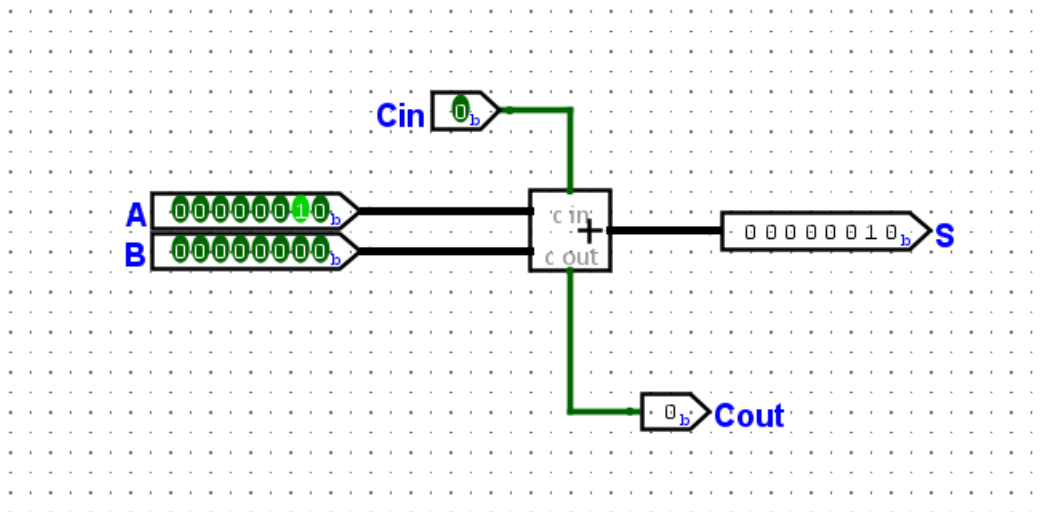
Creating Full-Adder using Logic Gates:



Truth Table:

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full-Adder using adder component with 8-bit input:



Conclusion:

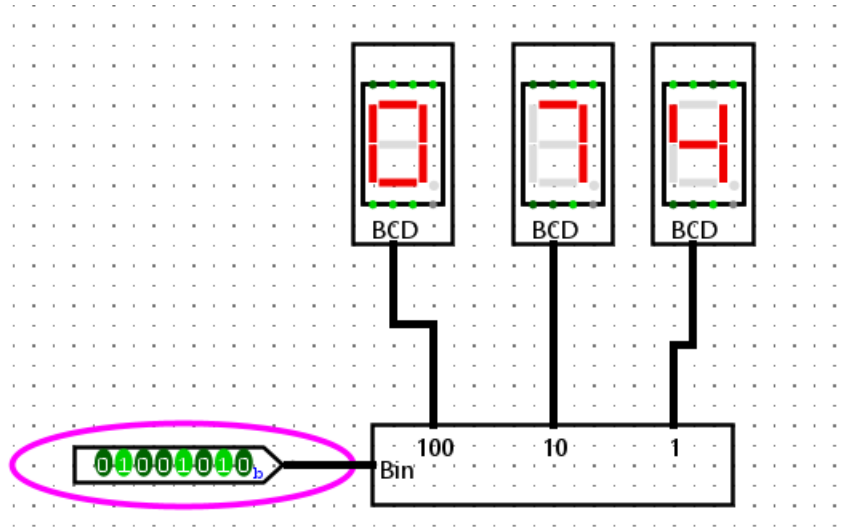
The Full Adder is a foundational digital circuit that performs complete 3-bit binary addition by calculating a Sum ($S = A \oplus B \oplus C_{in}$) and a Carry-Out ($C_{out} = AB + C_{in}(A \oplus B)$) for any given input combination of two data bits and an incoming carry bit. By evaluating the truth table, we observe that the Sum output functions as an odd-parity detector—yielding logic High whenever an odd number of inputs are active—while the Carry-Out acts as a majority voting function that produces a carry bit whenever two or more inputs are High. Unlike a Half Adder, this capability to process an incoming carry (C_{in}) allows multiple Full Adders to be cascaded in series, making them indispensable building blocks for constructing multi-bit adders, arithmetic logic units (ALUs), and complex computational processors.

BCD Converter:

A BCD (Binary Coded Decimal) Converter is a combinational logic circuit designed to translate data between standard binary notation—which digital microprocessors use natively for efficient arithmetic—and Binary Coded Decimal format, where each individual decimal digit (0 through 9) is independently represented by a 4-bit binary nibble (0000₂ to 1001₂). By bridging binary computing systems and human-readable interfaces, these converters enable seamless data communication, allowing binary calculation results to be formatted for human decimal displays like calculators and digital multimeters, or converting user keypresses into pure binary for CPU processing.

Required Components:

1. Input Pin (8-bit)
2. Binary-to-BCD Subcircuit / Module
3. Hex Displays (3×)



The Binary-to-BCD converter takes an 8-bit binary input (ranging from 0 to 255) and processes it—typically using the Shift-Add-3 (Double Dabble) algorithm—to separate the binary value into three 4-bit Binary Coded Decimal (BCD) nibbles representing the Hundreds, Tens, and Units places. As the binary bits are shifted through the logic, any 4-bit column evaluating to 5 or greater is increased by 3 before the next shift to prevent arithmetic overflow in decimal, yielding three distinct 4-bit outputs that drive the respective Hex Displays to show the exact decimal value (074 in this example).